

Tribhuvan University

Faculty of Humanities and Social Science

Damak Multiple Campus

TOUR KINGS

A Web-Based Tourism and Trek-Planning Platform with AI-Assisted Trek Recommendation

A PROJECT REPORT

Submitted to

Department of Computer Application

Damak Multiple Campus

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted by

Rojan Chandra Poudel (20202130)

Sushant Basnet (20202135)

Semester: 8th

Under the Supervision of

Mr. Abhinash Kumar Jha

September, 2026

Tribhuvan University
Faculty of Humanities and Social Science
Damak Multiple Campus

Supervisor's Recommendation

I hereby recommend that this project prepared under my supervision by Rojan Chandra Poudel and Sushant Basnet entitled "TOUR KINGS - A Web-Based Tourism and Trek-Planning Platform with AI-Assisted Trek Recommendation" in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

.....

SIGNATURE OF SUPERVISOR

Mr. Abhinash Kumar Jha

SUPERVISOR

Department of BCA

Damak Multiple Campus, Damak-9, Jhapa



Tribhuvan University
Faculty of Humanities and Social Science
Damak Multiple Campus

LETTER OF APPROVAL

This is to certify that this project prepared by Rojan Chandra Poudel and Sushant Basnet entitled "TOUR KINGS" in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

…… **SIGNATURE OF SUPERVISOR** Mr. Abhinash Kumar Jha **SUPERVISOR** Damak Multiple Campus Damak-9, Jhapa	…… **SIGNATURE OF CO-ORDINATOR** Mr. Abhinash Kumar Jha **BCA CO-ORDINATOR** Damak Multiple Campus Damak-9, Jhapa
…… **SIGNATURE OF INTERNAL EXAMINER**	…… **SIGNATURE OF EXTERNAL EXAMINER**

ABSTRACT

Trekking and tourism information in Nepal is still scattered across brochure-style agency websites, printed pamphlets and social-media pages, so a traveller who wants a trek matched to their available time, fitness and budget usually has to contact an agency and wait for a human reply. Tour Kings (TourApp) is a full-stack web platform built to close that gap: it consolidates destinations, tours and treks in one searchable catalogue and adds two AI-assisted features - a personalised trek-suggestion engine that filters and ranks treks against a traveller's stated preferences, and a conversational travel assistant, built on the Hugging Face inference API, that answers natural-language travel questions at any hour.

The system was implemented on the MERN stack: MongoDB for storage, Express.js and Node.js for the REST API, and React (bundled with Vite, styled with Tailwind CSS and animated with Framer Motion) for the single-page client. Authentication and role-based access control use JSON Web Tokens and bcrypt password hashing, separating traveller accounts from administrative accounts that manage content. The analysis and design followed object-oriented practice - use case, class, state, sequence, activity, and component and deployment models, supported by data-flow and database designs - to turn the requirements into a layered, maintainable architecture. Trek suggestion is deliberately two-stage: conventional rule-based filtering in testable backend code, followed by an AI-generated explanation, with a predefined fallback whenever the external service is slow or unreachable.

The system was verified by unit testing of the API endpoints and system testing of the end-to-end workflows. All test cases passed, and the fallback path returned a usable response within the normal timeout when the AI service was deliberately disabled, showing that a modest integration of an existing inference service can add meaningful personalisation without training a custom model. The report closes with the current limitations - no online payment, one interface language and a dependency on a third-party API - and recommends payment integration, reviews, offline (PWA) support and multi-language capability as the next steps. Both authors contributed to every stage.

Keywords: tourism platform, trek recommendation, MERN stack, AI chatbot, JWT authentication, Nepal trekking.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our supervisor and BCA Programme Co-ordinator, Mr. Abhinash Kumar Jha, for his continuous guidance, technical insight and patient review of every increment of this work. His suggestions on scoping the AI-assisted features, and on grounding every recommendation in explainable, verifiable data, shaped the direction of the entire project.

We also thank all our teachers at Damak Multiple Campus, whose teaching over the past eight semesters gave us the foundation in programming, databases, software engineering and artificial intelligence on which this project rests, and the Department of Computer Application for the laboratory facilities and academic environment that made the work possible. We are grateful to Tribhuvan University and the Faculty of Humanities and Social Science for a curriculum that lets students build a complete, working software system as part of their degree.

Finally, we thank our classmates and friends who volunteered as test users, patiently chatted with early and often unhelpful versions of the assistant and gave the honest feedback that exposed several defects which would otherwise have reached this report, and our families for their constant encouragement.

Rojan Chandra Poudel
Symbol No: 20202130

Sushant Basnet
Symbol No: 20202135

TABLE OF CONTENTS

Chapter 1: Introduction 1

1.1 Introduction 1

1.2 Problem Statement 1

1.3 Objectives 2

1.4 Scope and Limitation 2

1.4.1 Scope 2

1.4.2 Limitations 3

1.5 Development Methodology 3

1.6 Report Organization 4

Chapter 2: Background Study and Literature Review 5

2.1 Background Study 5

2.1.1 The MERN Stack 5

2.1.2 REST APIs and Client–Server Architecture 5

2.1.3 Authentication, Authorisation and Password Security 5

2.1.4 Natural Language Processing and AI-Assisted Recommendation 6

2.2 Literature Review 6

2.2.1 Review of Existing Systems 6

2.2.2 Review of Related Literature 7

2.2.3 Research Gap 7

Chapter 3: System Analysis and Design 9

3.1 System Analysis 9

3.1.1 Requirement Analysis 9

3.1.2 Feasibility Analysis 11

3.1.3 Object Modelling using Class and Object Diagrams 12

3.1.4 Dynamic Modelling using State and Sequence Diagrams 13

3.1.5 Process Modelling using Activity Diagrams 15

3.2 System Design 16

3.2.1 Refinement of the Analysis Models 16

3.2.2 Component Diagram 19

3.2.3 Deployment Diagram 20

3.3 Algorithm Details 21

Chapter 4: Implementation and Testing 23

4.1 Implementation 23
4.1.1 Tools Used 23
4.1.2 Implementation Details of Modules 23
4.2 Testing 25
4.2.1 Test Cases for Unit Testing 26
4.2.2 Test Cases for System Testing 26
4.3 Result Analysis 27
Chapter 5: Conclusion and Future Recommendations 29
5.1 Conclusion 29
5.2 Future Recommendations 29
References 31
Appendix A: Screenshots of the System 32

LIST OF FIGURES

Figure 1.1: The incremental development model followed in this project 4
Figure 3.1: Use case diagram of the Tour Kings system 10
Figure 3.2: Gantt chart of the project schedule 12
Figure 3.3: Class diagram of the Tour Kings system 13
Figure 3.4: State diagram of the booking lifecycle 14
Figure 3.5: Sequence diagram of a personalised trek suggestion 15
Figure 3.6: Activity diagram of the tour booking process 16
Figure 3.7: System architecture of the Tour Kings platform 17
Figure 3.8: Level 0 data flow diagram (context diagram) 17
Figure 3.9: Level 1 data flow diagram 18
Figure 3.10: Database design: collections and their relationships 19
Figure 3.11: Component diagram of the Tour Kings system 20
Figure 3.12: Deployment diagram of the Tour Kings system 21
Figure 3.13: The two-stage trek suggestion algorithm 22
Figure 4.1: The trek suggester: preference form and ranked suggestions 25
Figure 4.2: The AI travel assistant answering a natural-language query 25
Figure A.1: Home page with the hero section and featured treks 32
Figure A.2: Trek catalogue with difficulty, duration and budget filters 32
Figure A.3: Trek details page with the booking action 33
Figure A.4: My Bookings page showing each request and its status 33
Figure A.5: Administrative dashboard with platform metrics and booking management 34

LIST OF TABLES

Table 2.1: Comparison of existing systems with the proposed system 6
Table 3.1: Use case overview 10
Table 3.2: Non-functional requirements 11
Table 3.3: Economic feasibility - indicative cost estimate (NPR) 12
Table 3.4: MongoDB collections and their purpose 19
Table 4.1: Tools and technologies used 23
Table 4.2: Implemented modules and their responsibilities 23
Table 4.3: Unit test cases 26
Table 4.4: System test cases 26
Table 4.5: Result analysis summary 27

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
BCA	Bachelor of Computer Application
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
HTTP/HTTPS	Hyper Text Transfer Protocol (Secure)
JSON	JavaScript Object Notation
JWT	JSON Web Token
MERN	MongoDB, Express.js, React.js, Node.js
NLP	Natural Language Processing
ODM	Object Data Modeling
PWA	Progressive Web Application
REST	Representational State Transfer
SPA	Single Page Application
UI	User Interface
UML	Unified Modeling Language

Chapter 1: Introduction

1.1 Introduction

Tourism is one of the largest contributors to Nepal's national economy, and trekking tourism in particular - centred on the Everest, Annapurna and Langtang regions - draws a significant share of international visitors every year. Despite that economic importance, most of the information a traveller needs in order to plan a trek (route difficulty, duration, seasonal suitability, estimated cost and required permits) is still scattered across static brochure-style websites, printed pamphlets and informal word-of-mouth channels run by individual agencies. A traveller who wants a trek matched to their available time, fitness level, interests and budget generally has to contact an agency directly and wait for a human response, which does not scale and is unavailable outside business hours.

Tour Kings, referred to in the code base as TourApp, is a web-based tourism and trek-planning platform developed to close that gap. It is built on the MERN stack (MongoDB, Express.js, React.js and Node.js) and combines a conventional content-browsing experience - destinations, tours and trek listings - with two AI-assisted features: a personalised trek-suggestion module that filters and ranks the available treks against a set of user-supplied preferences, and a conversational travel assistant, powered by the Hugging Face inference API, that answers open-ended natural-language questions about destinations and travel logistics at any time of day. Secure authentication using JSON Web Tokens and role-based access control give travellers and administrators appropriately scoped permissions.

This report documents the analysis, design, implementation and testing of the system as a final-year Project III submission for the Bachelor of Computer Application programme at Damak Multiple Campus, Tribhuvan University.

1.2 Problem Statement

Independent travellers planning a visit to Nepal face four recurring difficulties:

- Trek-planning information - cost, duration, difficulty and best season - is spread across many independent websites and social-media pages with no consistent structure, which makes comparing options slow and error-prone.

- Existing agency websites are largely static catalogues; they do not adapt what they show to an individual traveller's stated duration, difficulty preference, interests or budget.
- Support for travel queries is normally limited to office hours and depends on a human agent being free, leaving international visitors in other time zones without timely answers.
- Booking and enquiry workflows are still manual - phone calls or e-mail - with no central, persistent record of a traveller's booking history.

Tour Kings reduces these frictions by consolidating tour and trek content in one searchable platform, adding preference-based trek matching and providing an always-available AI assistant, while degrading gracefully to predefined content whenever the external AI service is unreachable.

1.3 Objectives

The objectives of this project are:

- To design and develop a full-stack web application that lets travellers browse destinations, places and treks through a single unified interface.
- To implement a personalised trek-suggestion feature that filters and ranks trek options according to a traveller's stated duration, difficulty, interests and budget.
- To integrate a conversational AI travel assistant capable of answering natural-language travel queries, with a predefined fallback response when the AI service is unavailable.
- To implement secure authentication and role-based authorisation using JWT and bcrypt.
- To provide an administrative interface for managing tour, trek and place content.
- To verify the correctness and reliability of the system through unit and system testing.

1.4 Scope and Limitation

1.4.1 Scope

- Browsing of tours, treks and places with descriptive content and images.
- User registration, login and profile management secured with JWT-based sessions.
- A trek-preference form that returns ranked, personalised trek suggestions.

- An AI-powered chat assistant for open-ended travel queries.
- A booking-enquiry workflow that stores requests for administrative follow-up.
- An administrative dashboard for managing tour, trek and place records and reviewing bookings.

1.4.2 Limitations

- The system does not process online payments; bookings are recorded as enquiries and confirmed manually by an administrator.
- The AI-assisted features depend on the availability and response quality of the third-party Hugging Face inference API; when it is unavailable the system falls back to static predefined responses rather than a locally hosted model.
- The platform supports a single interface language (English) and does not yet provide offline (PWA) access.
- Trek-suggestion ranking uses rule-based filtering over structured trek attributes rather than a trained recommendation model, so its accuracy is bounded by the completeness of the underlying trek data.

1.5 Development Methodology

The project followed an iterative and incremental, Agile-inspired methodology rather than a single-pass waterfall process, because the requirements for the AI-assisted features benefited from repeated refinement once early prototypes had been tested against real trek data. The work was organised into three increments aligned with the phases mandated by the Project III guidelines - proposal, mid-term progress and final defence - each ending in a working, demonstrable slice of the system that the supervisor reviewed before the next increment started.

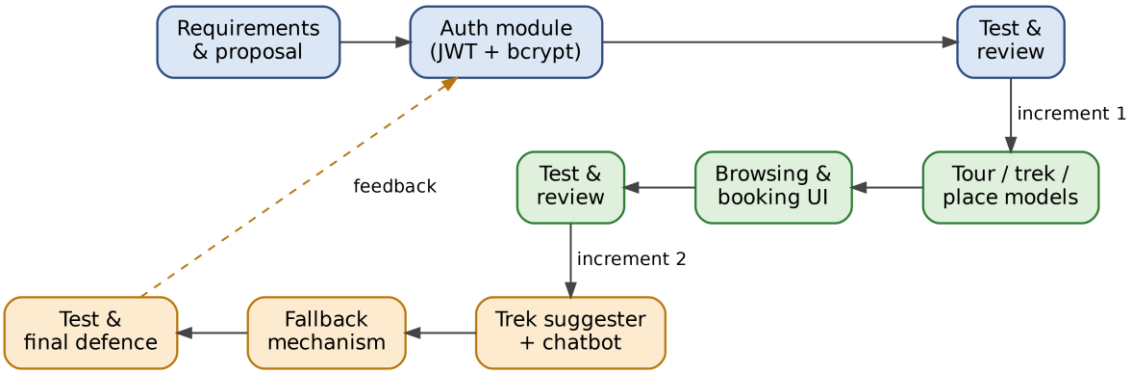


Figure 1.1: The incremental development model followed in this project

The incremental approach proved its value directly: the fallback mechanism for the AI features was added only after testing of the second increment revealed that the third-party inference API could occasionally be slow or unreachable, a risk that a strict upfront design would not have surfaced until much later.

1.6 Report Organization

The rest of this report is organised into four chapters. Chapter 2 presents the background theory behind the technologies used and reviews existing systems and related literature on AI-assisted tourism chatbots and recommender systems. Chapter 3 describes the system analysis and design, covering requirement analysis, feasibility, the UML models, the data-flow and database designs, and the trek-suggestion algorithm. Chapter 4 describes the implementation of each module, the tools used and the unit and system testing performed, together with an analysis of the results. Chapter 5 concludes the report and recommends future enhancements.

Chapter 2: Background Study and Literature Review

2.1 Background Study

2.1.1 The MERN Stack

MERN is a JavaScript-based full-stack combination of MongoDB (a document-oriented NoSQL database), Express.js (a minimal web framework for Node.js used to build REST APIs), React.js (a component-based library for interactive user interfaces) and Node.js (the server-side JavaScript runtime). Because all four layers use one language, data structures and validation logic can be shared between client and server, which reduces context-switching compared with stacks that mix languages. In this project React, bundled with Vite, renders the single-page client; Express and Node implement the REST API and business logic; and MongoDB, accessed through the Mongoose object-data-modelling library, stores flexible JSON-like documents for users, tours, places and bookings, which suits the semi-structured nature of trek and destination content.

2.1.2 REST APIs and Client–Server Architecture

The Representational State Transfer style structures client–server communication as stateless HTTP requests against uniquely addressable resources such as /api/tours or /api/bookings, using the standard verbs GET, POST, PUT and DELETE to express intent. The backend of this system exposes a set of REST routes - authRoutes, tourRoutes and aiRoutes among others - that the React frontend consumes through the Axios HTTP client, keeping the presentation layer and the business and data layers independently deployable and testable.

2.1.3 Authentication, Authorization and Password Security

JSON Web Tokens are a compact, URL-safe way of representing claims such as a user's identity and role, digitally signed so that a server can verify their integrity without storing session state. On successful login the backend issues a signed JWT that the client attaches to later requests; middleware on protected routes verifies the token before granting access, and the role claim - traveler or administrator - determines which operations are authorized. Passwords are never stored in plain text: bcrypt applies a deliberately expensive, salted one-way hash, so even a database compromise does not directly expose user credentials.

2.1.4 Natural Language Processing and AI-Assisted Recommendation

Natural Language Processing is the branch of artificial intelligence concerned with interpreting and generating human language. Modern transformer-based models, including those hosted on the Hugging Face inference API used here, are pre-trained on large text corpora and can be reached through a single API call instead of being trained from scratch. This project uses that capability twice: to power a general conversational assistant, and to generate the descriptive justification attached to trek suggestions that have first been filtered by conventional rule-based matching against structured attributes stored in MongoDB. The hybrid arrangement keeps the core recommendation logic predictable while still giving the responses a conversational quality.

2.2 Literature Review

2.2.1 Review of Existing Systems

Three categories of system already serve parts of this problem: general online travel agencies, Nepali trekking-agency websites and generic AI chat assistants. Table 2.1 compares them with the proposed system on the capabilities that matter for trek planning.

Table 2.1: Comparison of existing systems with the proposed system

Capability	Online travel agencies	Nepali agency websites	General AI assistants	Tour Kings (proposed)
Structured trek attributes (altitude, season, difficulty)	Partial	Yes	No	Yes
Preference-based ranking of treks	Generic	No	Unreliable	Yes
24 × 7 natural-language answers	No	No	Yes	Yes
Answers grounded in a verified trek catalogue	Yes	Yes	No	Yes
Graceful fallback when the AI service is unavailable	-	-	No	Yes
Booking record kept for the user	Yes	No	No	Yes
Focus on the Nepal trekking context	No	Yes	No	Yes

General agencies rank hotels and flights rather than treks and know nothing about altitude or seasonal accessibility; agency websites hold that knowledge but present it as a fixed catalogue; and a general assistant answers freely but is not grounded in any verified inventory. The proposed system is the only one of the four that combines a trek-attribute-aware ranking with a conversational layer over the same verified data.

2.2.2 Review of Related Literature

Camilleri and Troise [1] carried out a systematic review of chatbot recommender systems in travel, tourism and hospitality. They report that chatbot features are generally seen as improving the service quality of online travel businesses and the resulting consumer experience, but also that current-generation chatbots are not yet a substitute for a human agent once a query becomes complex. That finding directly motivated the decision in this project to keep a predefined fallback path alongside the AI-generated one instead of relying on the inference service exclusively.

Alotaibi et al. [2] present an AI chatbot built to generate tourism recommendations for visitors to Jeddah, showing that a chat interface can act not only as a question-answering channel but as the primary route through which a visitor receives destination recommendations - a role analogous to the trek suggester here, although this project also exposes the same capability as a structured preference form. Saluja et al. [3] describe an AI-based travel chatbot for content-oriented queries and reinforce that combining retrieval of structured content with a conversational layer is a recurring, practical pattern rather than an approach unique to this work. Applied studies of the MERN stack [4], [5] report that the shared JavaScript runtime reduces developer context-switching and simplifies reuse of validation and data-shape logic, which matched this project's own experience: the Mongoose schema definitions used by the backend informed the shape of the data consumed by the React components and reduced integration errors between the layers.

2.2.3 Research Gap

Both AI-assisted tourism chatbots and MERN-based web platforms are individually well established, but most published tourism-chatbot case studies target general international destinations or single cities, and comparatively little work addresses Nepal's trekking-tourism context, where altitude, seasonal accessibility and physical difficulty are decisive in matching a trek to a traveler's ability and available time. This project addresses that gap by combining a structured, trek-attribute-aware suggestion engine with a general-purpose AI assistant inside a single regionally focused platform, instead of treating either capability in isolation.

Chapter 3: System Analysis and Design

3.1 System Analysis

3.1.1 Requirement Analysis

Requirements were gathered by studying comparable trekking-agency and tourism-board websites, reviewing the functionality implied by the approved proposal and refining the feature set during the mid-term development cycle. They are grouped below into functional requirements - what the system must do - and non-functional requirements, the qualities it must exhibit.

i. Functional Requirements

- A visitor shall be able to register an account and log in with an e-mail and password.
- The system shall issue a signed JWT on successful login and use it to authorize later requests.
- Any visitor shall be able to browse tours, treks and places without logging in.
- A logged-in user shall be able to submit trek preferences - duration, difficulty, interests and budget - and receive a ranked list of matching trek suggestions.
- The system shall provide a chat interface that returns an AI-generated response to a natural-language travel query, or a predefined fallback response if the AI service is down.
- A logged-in user shall be able to submit a booking or enquiry request for a tour or trek.
- An administrator shall be able to create, update and delete tour, trek and place records.
- An administrator shall be able to view registered users and manage submitted bookings.

These requirements are illustrated by the use case diagram in Figure 3.1, which identifies four actors: the unauthenticated Visitor, the Registered User, the Administrator and the external AI chat service. Table 3.1 summarizes the primary use cases and the actors that initiate them.

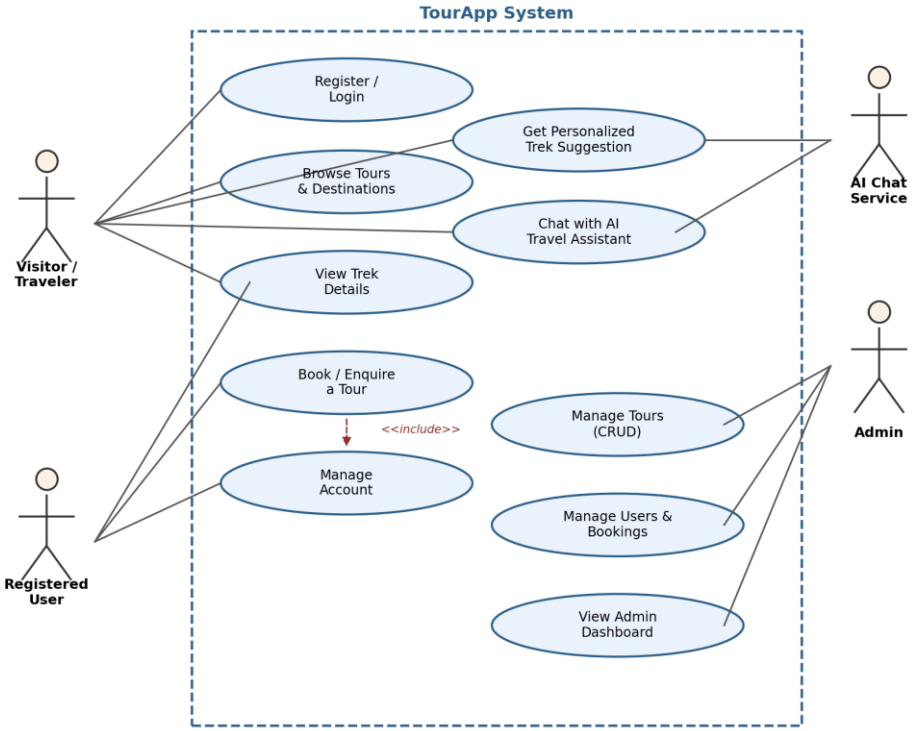


Figure 3.1: Use case diagram of the Tour Kings system

Table 3.1: Use case overview

Use case	Primary actor	Description
Register / Login	Visitor	Create an account or authenticate an existing one; a JWT is issued on success.
Browse tours& destinations	Visitor / Registered User	View listed tours, treks and places with descriptions and images.
Get personalised trek suggestion	Registered User	Submit preferences and receive ranked, matching trek suggestions.
Chat with the AI travel assistant	Visitor / Registered User	Send a natural-language query and receive an AI or fallback response.
Book / enquire about a tour	Registered User	Submit a booking or enquiry request for a specific tour or trek.
Manage tours (CRUD)	Admin	Create, update and delete tour, trek and place records.
Manage users& bookings	Admin	Review registered users and submitted booking requests.

ii. Non-Functional Requirements

Table 3.2: Non-functional requirements

Quality	Requirement
Security	Passwords must be stored as bcrypt hashes; protected API routes must reject any request without a valid JWT, and role checks must be enforced on the server.
Performance	Listing pages must respond within an acceptable time under normal load, helped by the asynchronous, non-blocking I/O model of Node.js.
Reliability	The AI-assisted features must degrade to predefined responses rather than fail when the external inference API is slow or unreachable.
Usability	The interface must be responsive across desktop and mobile viewports, using Tailwind CSS utilities and Framer Motion transitions.
Maintainability	The codebase must separate concerns across routes, models and middleware on the server and components, pages, context and services on the client.

3.1.2 Feasibility Analysis

i. Technical Feasibility

Every technology required - Node.js, Express, MongoDB, React, JWT, bcrypt and the Hugging Face inference API - is freely available, well documented and already familiar from earlier coursework, so the project was technically achievable within one semester using the hardware available in the campus laboratory.

ii. Operational Feasibility

The system needs only a modern web browser and no specialised hardware, so it can realistically be adopted by travellers and by a small agency administrative team without training beyond basic web navigation.

iii. Economic Feasibility

Development relies entirely on open-source frameworks, and the inference API offers a free tier sufficient for a project of this scale, so the cost is essentially the time invested. Table 3.3 gives an indicative estimate had the same work been costed commercially.

Table 3.3: Economic feasibility - indicative cost estimate (NPR)

Item	Basis	Cost
Development effort	18 weeks × 2 students, valued at student rate	- (academic)
Domain name	.com.np registration, one year	1,800
Cloud hosting	Application server and static hosting, free tier	0

Item	Basis	Cost
MongoDB Atlas	Shared M0 cluster, free tier	0
Hugging Face inference	Free inference tier within rate limits	0
Contingency	Paid tier if request volume grows	12,000
Total recurring cost for the pilot year		13,800

iv. Schedule Feasibility

The feature set was implementable within the phased Project III timeline - proposal by week 3, mid-term defence by week 12 and final defence before the board examination - because the incremental method completed the core browsing features early and left the later iterations for the more experimental AI-assisted modules. Figure 3.2 shows the schedule actually followed.

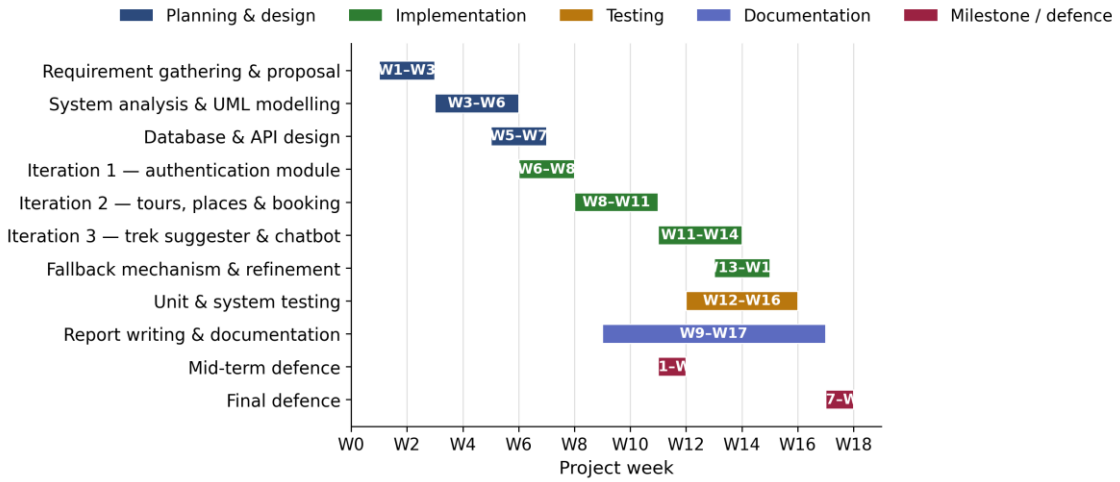


Figure 3.2: Gantt chart of the project schedule

3.1.3 Object Modelling using Class and Object Diagrams

The core domain model comprises six classes: User, Tour/Trek, Booking, Place, TrekPreference and ChatSession. A User browses Tour/Trek and Place records, submits a TrekPreference that generates suggestions, and can create one or more Bookings against a chosen tour; a ChatSession holds the message history of a user's interaction with the assistant. This structure was translated directly into Mongoose schemas, with ObjectId references representing the relationships between collections - a Booking document, for example, references both a userId and a tourId.

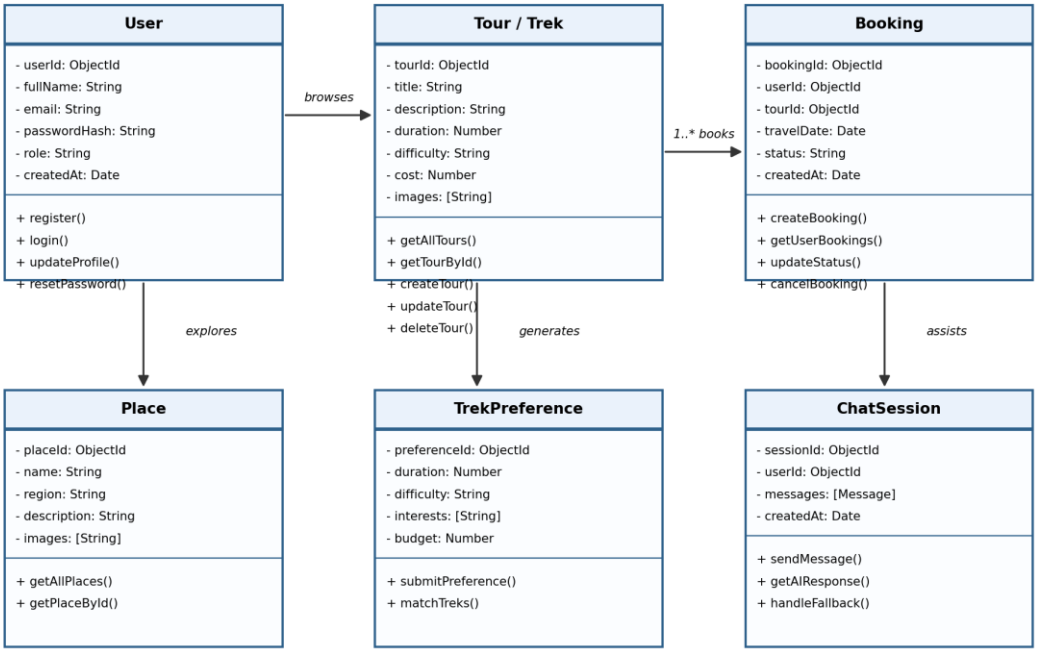


Figure 3.3: Class diagram of the Tour Kings system

3.1.4 Dynamic Modelling using State and Sequence Diagrams

The lifecycle of a Booking determines which actions are available to a user or administrator at any moment, and is modelled as the state machine in Figure 3.3. A booking enters the Pending state when the user submits an enquiry; an administrator may confirm it, after which it completes once the travel date passes; either party may cancel it before travel. Completed and cancelled are terminal states.

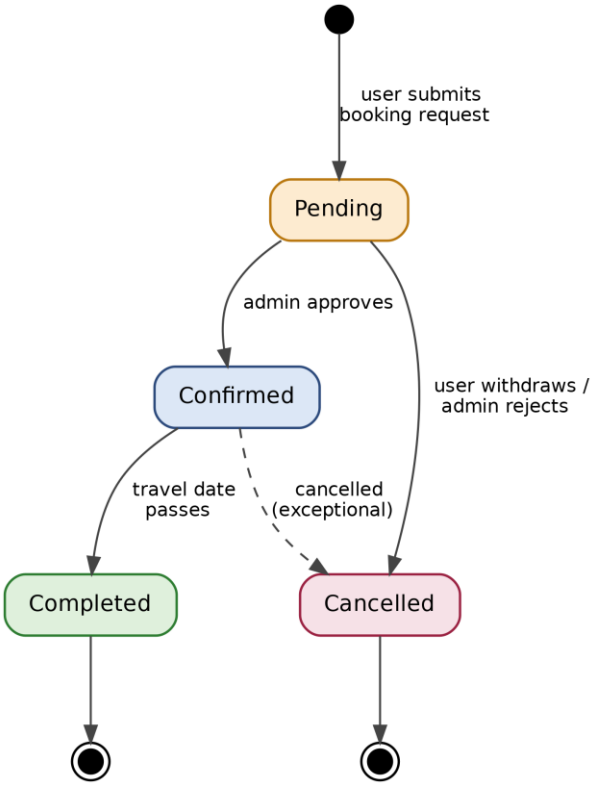


Figure 3.4: State diagram of the booking lifecycle

Figure 3.4 shows the interaction that takes place during a personalised trek suggestion. When the user submits preferences, the backend first queries MongoDB for treks matching the structured attributes, then asks the inference API to generate a natural-language ranking or description for the matched records; if that call fails or times out, the backend returns a predefined fallback suggestion so the user is never left without a response.

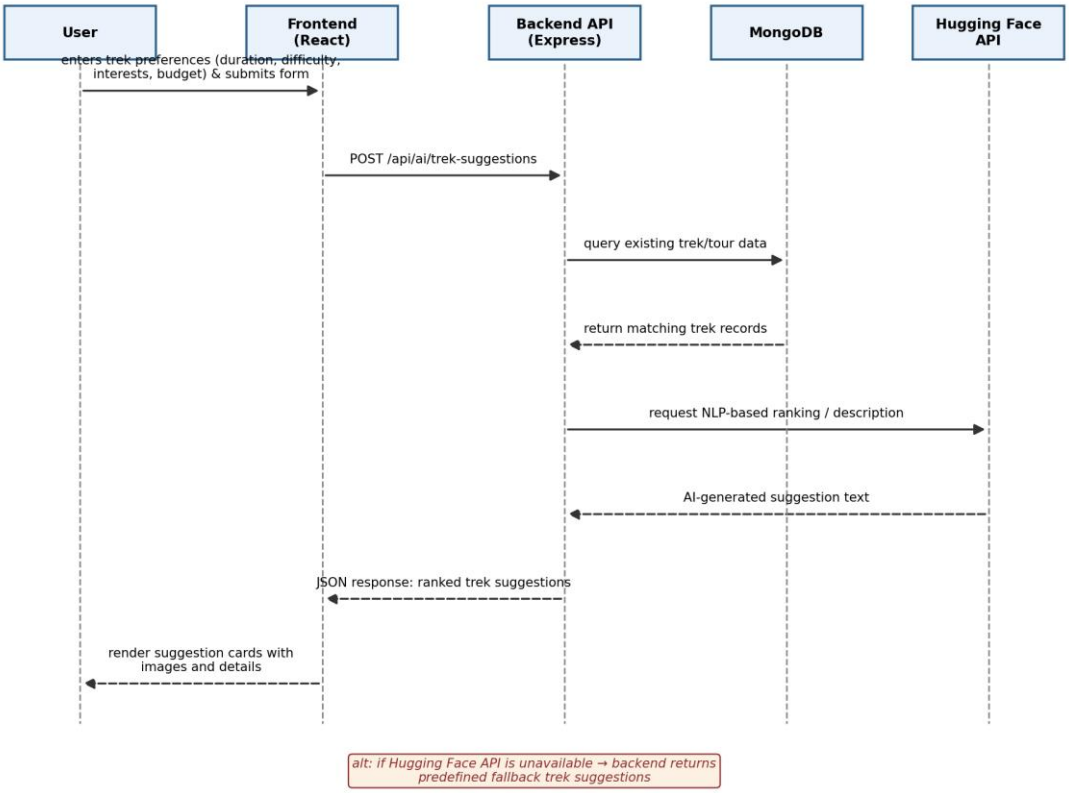


Figure 3.5: Sequence diagram of a personalised trek suggestion

3.1.5 Process Modelling using Activity Diagrams

Figure 3.5 models the end-to-end booking process as an activity diagram, from browsing tours through to a confirmed booking. It highlights the two decision points central to the workflow: whether the visitor is logged in before the booking form can be submitted, and whether the submitted form passes server-side validation before the record is persisted.

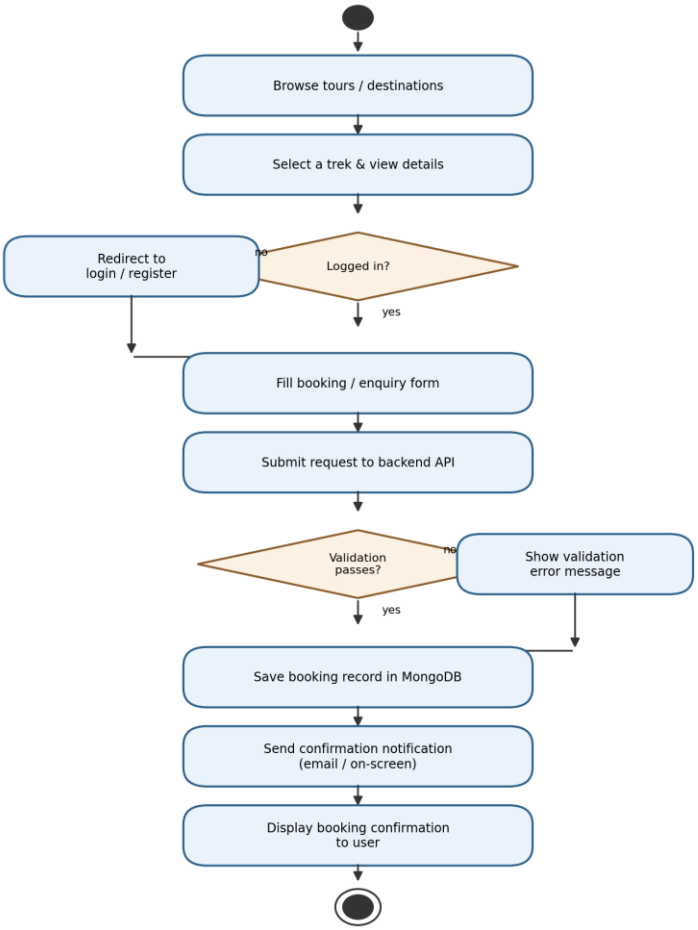


Figure 3.6: Activity diagram of the tour booking process

3.2 System Design

3.2.1 Refinement of the Analysis Models

During design refinement the abstract TrekPreference class was mapped to a transient request payload rather than a persisted collection, since preferences are needed only to compute a suggestion at request time; this removed unnecessary writes to MongoDB. ChatSession was refined to store only the recent exchange needed to render the chat window rather than an unbounded history, keeping document sizes predictable. Booking retained its full attribute set, because its status field drives the state model in Figure 3.3 and its history is a required administrative record.

i. System Architecture

The deployed system is organised as three layers - a React single-page client, an Express/Node REST API with authentication middleware, and a MongoDB persistence layer - with the Hugging Face inference API attached to the API layer as an external service.

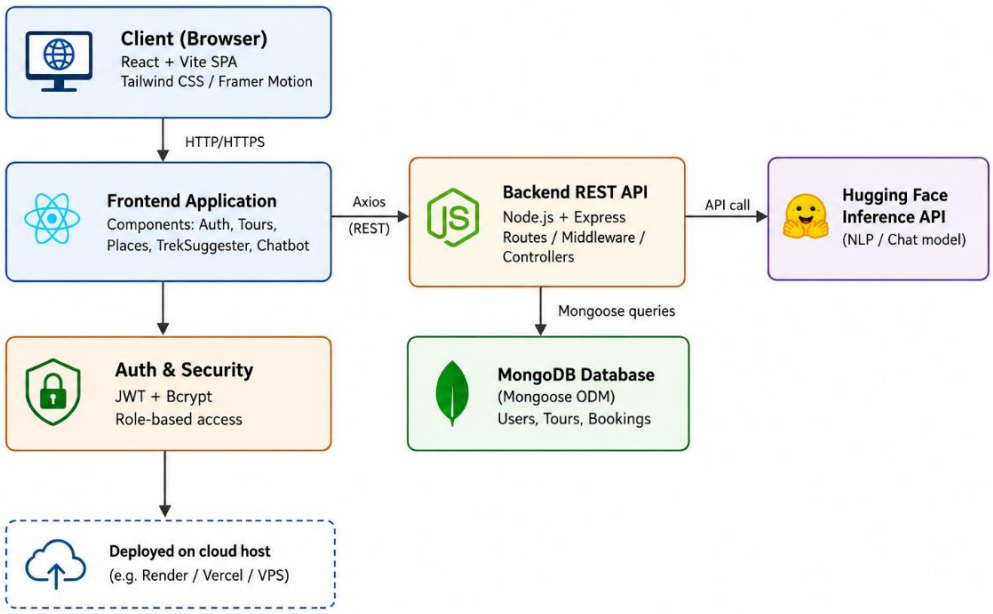


Figure 3.7: System architecture of the Tour Kings platform

ii. Process Model - Data Flow Diagrams

The context diagram in Figure 3.7 shows the system as a single process exchanging data with three external entities, and Figure 3.8 decomposes it into the five processes and three data stores that make up the platform.

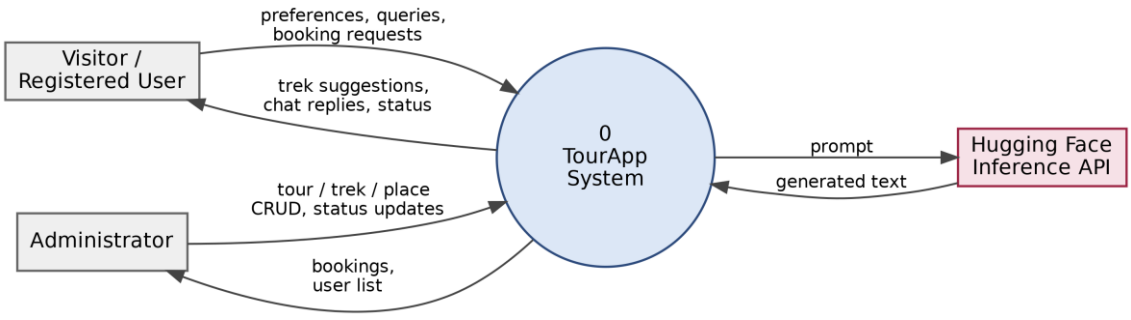


Figure 3.8: Level 0 data flow diagram (context diagram)

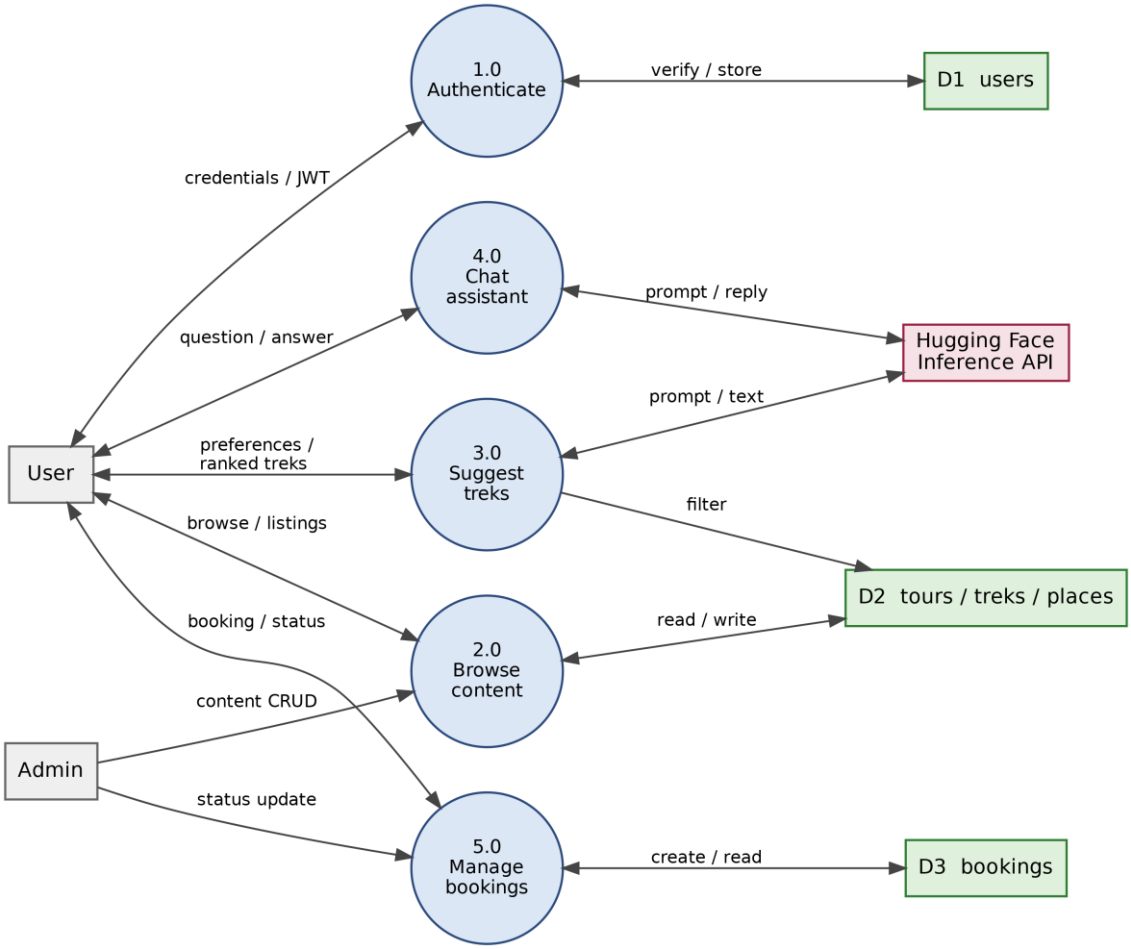


Figure 3.9: Level 1 data flow diagram

iii. Database Design

MongoDB stores five collections. Relationships are expressed as ObjectId references rather than joins, and the two AI features read from the same tours collection that the browsing pages use, so a suggestion can never point at a trek that is not in the catalogue.

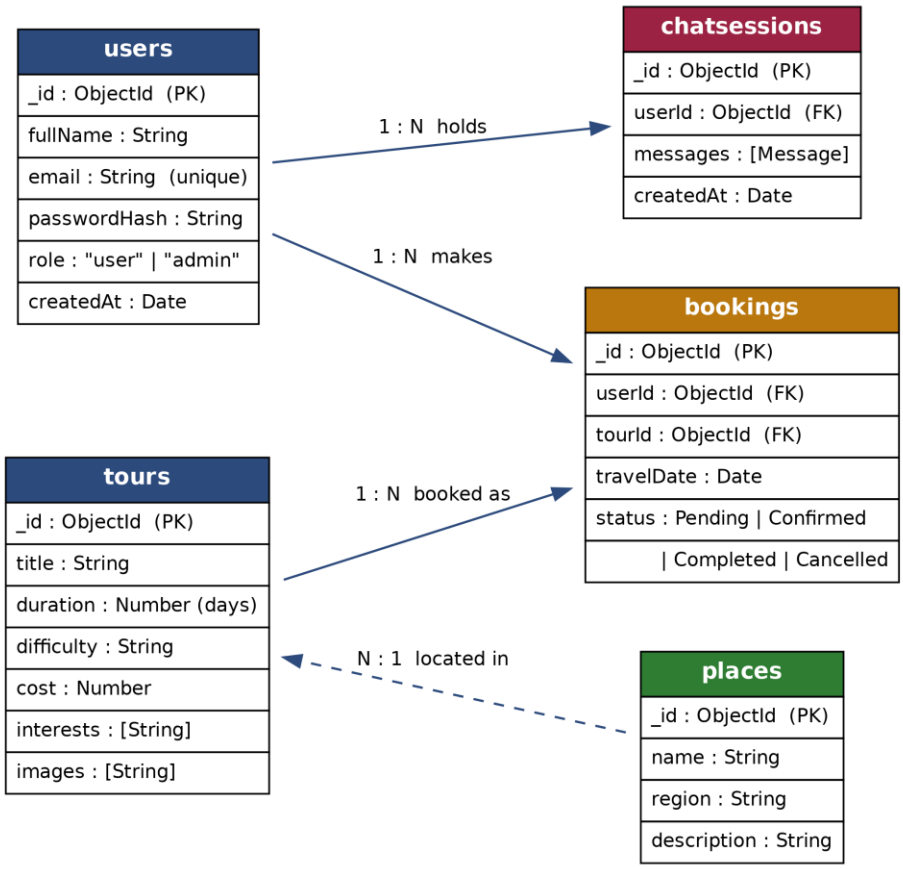


Figure 3.10: Database design: collections and their relationships

Table 3.4: MongoDB collections and their purpose

Collection	Key fields	Purpose
users	email, passwordHash, role	Account records; the role field drives access control.
tours	duration, difficulty, cost, interests	Tour and trek catalogue; the attributes used by the suggestion filter.
places	name, region, description	Destination content browsed independently of a trek.
bookings	userId, tourId, travelDate, status	Booking enquiries and the state of each one.
chatsessions	userId, messages	Recent chat exchange rendered in the assistant window.

3.2.2 Component Diagram

Figure 3.10 shows how the frontend feature components - Auth, Tours and Places, TrekSuggester and Chatbot - map onto the Express routers, and how each router depends on a supporting component: authRoutes.js on the JWT-verification middleware, tourRoutes.js on the Mongoose models, and aiRoutes.js on a wrapper service around the

inference API. The decomposition keeps each frontend feature isolated from the implementation details of the corresponding backend service.

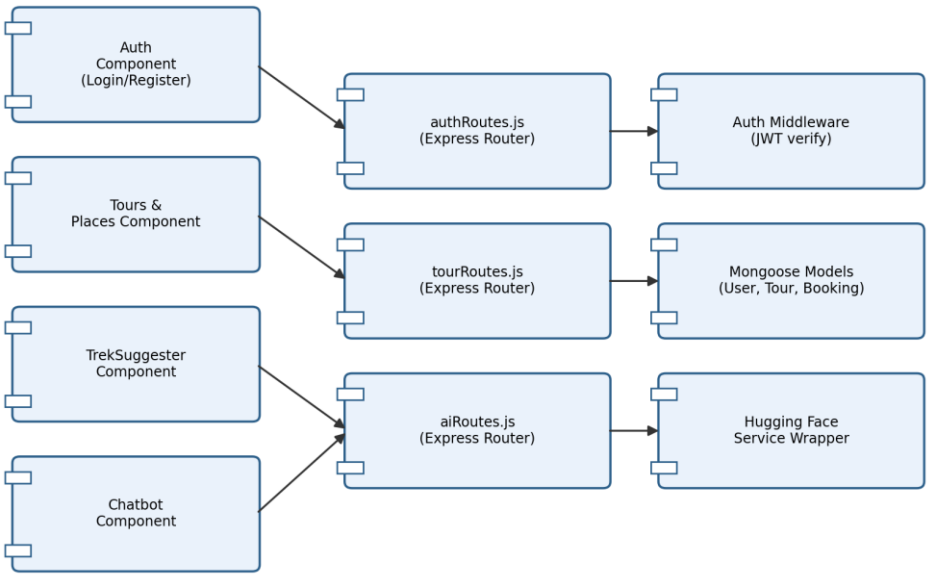


Figure 3.11: Component diagram of the Tour Kings system

3.2.3 Deployment Diagram

Figure 3.11 shows the physical topology: a client device running the React application in a browser, communicating over HTTPS with an application server that hosts the Node/Express API and the JWT middleware; the application server talks to a MongoDB database server over the MongoDB wire protocol and to the externally hosted inference API over HTTPS with an API key. Application server, database and AI provider can therefore be scaled or relocated independently of one another.

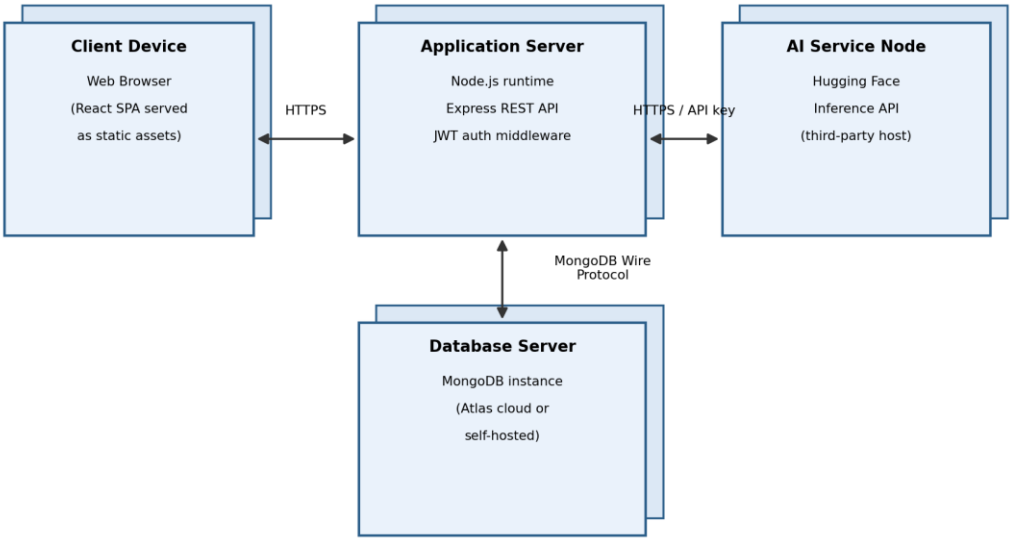


Figure 3.12: Deployment diagram of the Tour Kings system

3.3 Algorithm Details

Trek suggestion is implemented as a two-stage algorithm rather than a single opaque AI call, so that its behaviour stays predictable and explainable. Stage 1 queries the tours collection for records whose duration falls within the requested range, whose difficulty matches or is easier than the stated preference and whose cost does not exceed the budget, then scores each surviving record by counting how many of the user's interests intersect its tagged attributes. If nothing survives, the budget and difficulty bounds are relaxed by one step and the query is repeated. Stage 2 passes the scored candidates to the inference API, which generates a short natural-language justification for the top matches; if that call fails or times out, the backend attaches a predefined description template instead and returns the same score-ranked list.

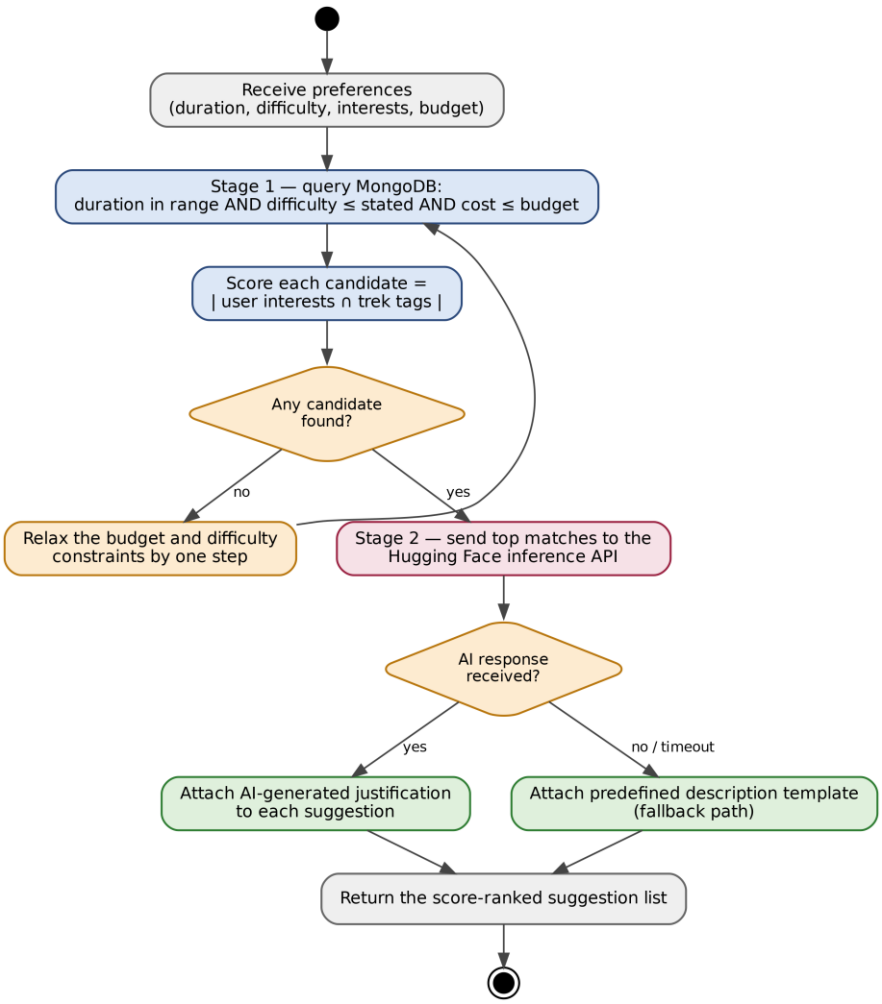


Figure 3.13: The two-stage trek suggestion algorithm

Keeping the filtering stage in conventional, testable backend code makes the suggestion behavior reproducible and easy to unit test - cases UT-08 and UT-09 in Section 4.2.1 produce the same candidate set whether or not the AI service responds - while still allowing the model to add a conversational explanation on top.

Chapter 4: Implementation and Testing

4.1 Implementation

4.1.1 Tools Used

Table 4.1: Tools and technologies used

Category	Tool / technology	Purpose
Frontend	React with Vite	Component-based single-page application and fast builds.
Styling	Tailwind CSS	Utility-first responsive styling.
Animation	Framer Motion, AOS	Page and scroll-triggered transitions.
HTTP client	Axios	REST requests from the client to the API.
Routing	React Router	Client-side navigation between pages.
Notifications	React Toastify	In-app success and error messages.
Backend	Node.js, Express.js	REST API server, routing and middleware.
Database	MongoDB, Mongoose	Document storage and object-data modelling.
Authentication	JWT, bcrypt	Token issue and verification; password hashing.
File upload	Multer	Image uploads for tour and place records.
E-mail	Nodemailer	Booking notification and confirmation mail.
AI integration	Hugging Face Inference API	Chat responses and suggestion descriptions.
Version control	Git and GitHub	Source versioning and collaboration.

4.1.2 Implementation Details of Modules

Table 4.2: Implemented modules and their responsibilities

Module	Server side	Responsibility
Authentication	authRoutes.js, auth middleware	Hashes the password with bcrypt on registration, compares it on login and signs a JWT carrying the user id and role. Middleware verifies the token on protected routes, rejects a missing or expired token with 401 and attaches the decoded user to the request for downstream role checks.
Tours, treks and places	tourRoutes.js, Mongoose models	Lists all records, fetches one by id and - for administrators only - creates, updates and deletes them. The client renders the results as animated cards; selecting one opens a details page with description, images, duration, cost and difficulty and a "Book This Trek" action.
Trek suggester	aiRoutes.js - trek-suggestions	Receives the preference payload, runs the two-stage algorithm of Section 3.3 and returns the score-ranked suggestions with an AI-generated or fallback justification. The client shows the form and the results in a tabbed interface.
Chatbot	aiRoutes.js - chat	Forwards the query with a short tourism-focused system prompt to the inference API and returns the reply. On failure

Module	Server side	Responsibility
		or timeout the exception is caught and one of a set of predefined fallback responses is returned, so the interface never surfaces a raw error.
Booking	bookingRoutes.js	Validates the payload, creates a Booking with status Pending, lets administrators move it through the states of Figure 3.3 and lets a user see the status of their own bookings from their profile page.
Admin dashboard	role-guarded routes	CRUD forms for tour, trek and place records including image upload via Multer, a list of registered users and a list of bookings whose status can be changed. Access is restricted on the client and, more importantly, enforced on the server, since a client-only restriction can be bypassed by calling the API directly.

Figures 4.1 and 4.2 show the two AI-assisted features in use. In Figure 4.1 the preference form on the left has produced a ranked list on the right, each entry carrying the generated justification for why it matches; in Figure 4.2 the assistant answers two consecutive travel questions in a single thread.

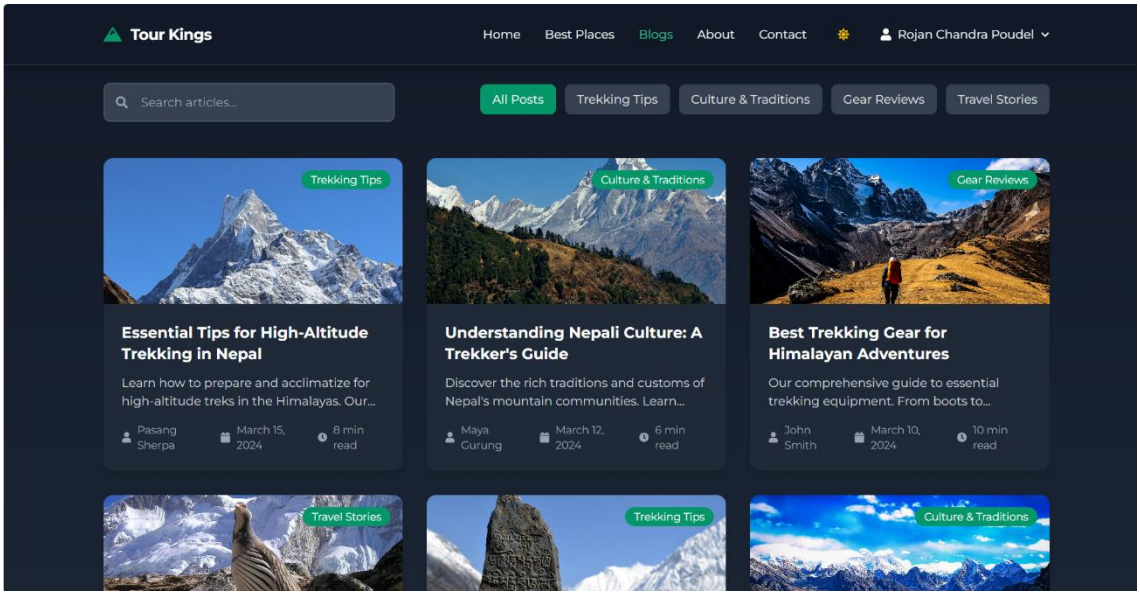


Figure 4.1: The trek suggested: preference form and ranked suggestions

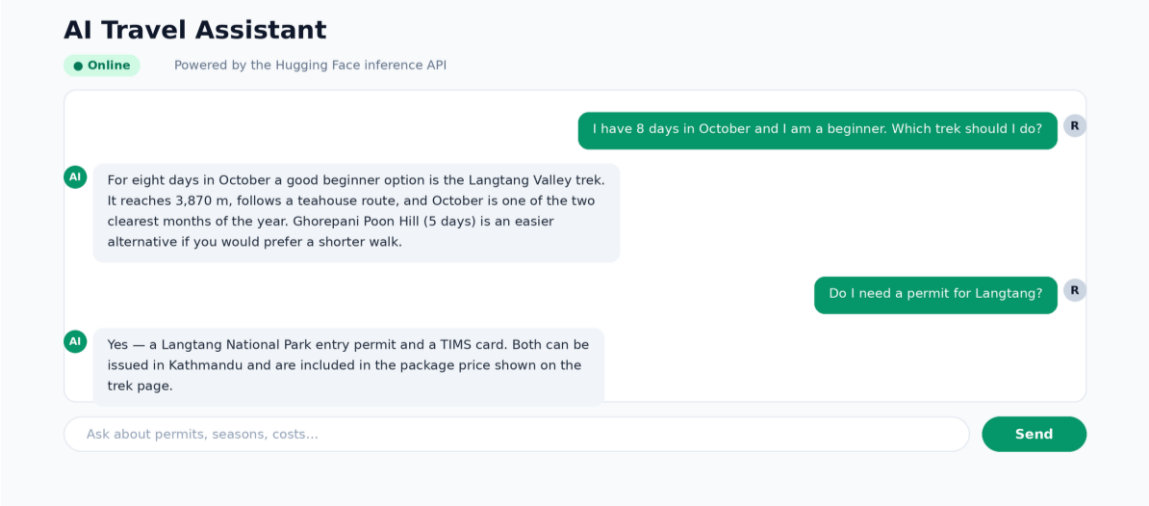


Figure 4.2: The AI travel assistant answering a natural-language query

4.2 Testing

Testing was carried out at two levels: unit testing of individual API endpoints in isolation, and system testing of complete end-to-end workflows spanning client and server together.

4.2.1 Test Cases for Unit Testing

Table 4.3: Unit test cases

ID	Endpoint	Input	Expected output	Result
UT-01	POST /api/auth/register	Valid new user details	201 Created; user saved with a hashed password	Pass
UT-02	POST /api/auth/register	E-mail already registered	400 Bad Request; duplicate-email error	Pass
UT-03	POST /api/auth/login	Correct e-mail and password	200 OK; signed JWT returned	Pass
UT-04	POST /api/auth/login	Incorrect password	401 Unauthorized; generic error	Pass
UT-05	GET /api/tours	No parameters	200 OK; JSON array of tour records	Pass
UT-06	POST /api/tours	Valid payload, admin token	201 Created; new tour returned	Pass
UT-07	POST /api/tours	Valid payload, traveler token	403 Forbidden; access denied	Pass
UT-08	POST /api/ai/trek-suggestions	Valid preferences, AI reachable	200 OK; ranked suggestions with AI description	Pass
UT-09	POST /api/ai/trek-suggestions	Valid preferences, AI disabled	200 OK; fallback suggestions, no 500 error	Pass
UT-10	POST /api/bookings	Valid payload, authenticated user	201 Created; booking saved as Pending	Pass

4.2.2 Test Cases for System Testing

Table 4.4: System test cases

ID	Scenario	Steps	Expected result	Result
ST-01	Registration and login	Register an account, then log in with the same credentials	User lands on the home page logged in, with a valid session token	Pass
ST-02	Browse and view a trek	Browse tours, then open a trek details page	Details, images, duration and cost are displayed correctly	Pass
ST-03	Personalized suggestion	Fill and submit the preference form as a logged-in user	A ranked list of matching treks with descriptions appears in the suggestions tab	Pass
ST-04	Chat with the assistant	Ask a travel question in the chat window	A relevant response appears in the thread within a reasonable time	Pass
ST-05	Chatbot fallback	Ask a question while the AI service is unreachable	A predefined fallback response is shown instead of an error page	Pass
ST-06	Submit a booking enquiry	Submit a booking request from a trek details page	Booking is recorded as Pending and appears on the profile page	Pass
ST-07	Administrator manages a tour	Log in as admin; create, edit and delete a tour record	The record is created, updated and removed; the public listing reflects each change	Pass
ST-08	Unauthorized access	Open an admin-only page as a regular traveler	Access is denied and the user is redirected away	Pass

4.3 Result Analysis

All eighteen test cases produced their expected results, indicating that the authentication, content-browsing, trek-suggestion, chat and booking workflows behave correctly under normal conditions. Table 4.5 summarizes the outcome against the objectives of Section 1.3.

Table 4.5: Result analysis summary

Objective	Evidence	Outcome
Unified browsing interface	ST-02; tours, treks and places served from one catalogue	Met
Preference-based trek ranking	UT-08, ST-03; ranked list returned for every valid preference set	Met
Conversational travel assistant	ST-04; relevant replies returned within the request timeout	Met
Graceful AI fallback	UT-09, ST-05; predefined response returned with no 5xx error	Met
Secure authentication and roles	UT-01 to UT-04, UT-07, ST-08; server-side role checks enforced	Met
Administrative content management	ST-07; full CRUD with changes visible in the public listing	Met
Online payment	Out of scope for this phase; bookings confirmed manually	Deferred

Three observations from testing are worth highlighting. First, role-based access control behaved correctly on both the intended path - an administrator performing a CRUD operation - and the negative path, confirming that authorization is enforced at the API layer rather than relying on client-side route restriction alone. Second, the fallback mechanism performed as designed: with the inference API deliberately made unreachable by using an invalid key, the system returned a predefined response inside the normal request timeout instead of surfacing a server error, meeting the reliability requirement of Table 3.2. Third, because the filtering stage runs independently of the AI call, its correctness could be verified deterministically, which validates the hybrid design discussed in Section 3.3. Overall the implemented system satisfies the functional and non-functional requirements identified in Chapter 3, within the scope and limitations stated in Section 1.4.

Chapter 5: Conclusion and Future Recommendations

5.1 Conclusion

This project set out to address the fragmented and impersonal way in which trekking and tourism information is presented to independent travelers planning a visit to Nepal. The resulting system, Tour Kings, consolidates tour, trek and destination content into a single MERN-stack web platform and augments it with two AI-assisted capabilities: a preference-driven trek-suggestion engine and a conversational travel assistant built on the Hugging Face inference API. Secure authentication and role-based authorization, implemented with JWT and bcrypt, separate ordinary traveler accounts from the administrative accounts that manage content.

The analysis and design applied standard object-oriented practice - use case, class, state, sequence, activity, and component and deployment models, supported by data-flow and database designs - to translate the identified functional and non-functional requirements into a layered, maintainable architecture. The deliberate hybrid design of the AI-assisted features, combining rule-based filtering with an AI-generated explanation and a predefined fallback path, kept the core behavior testable and reliable even when the third-party service was unavailable. Unit and system testing confirmed that authentication, browsing, trek suggestion, chat and booking all work as intended and that access control is correctly enforced at the API layer.

In summary, the project demonstrates that meaningful personalization can be added to a regional tourism platform through a modest, well-scoped integration of an existing inference service, without the resources needed to train a custom machine learning model. The objectives set out in Section 1.3 have been substantially achieved within the scope and limitations described in Section 1.4.

5.2 Future Recommendations

- Payment integration: add an online payment gateway so bookings can be paid for and confirmed automatically instead of requiring manual administrative confirmation.
- Reviews and ratings: let travellers who have completed a trek leave a rating and review, which would also give the suggestion engine an additional structured signal to rank on.

- Offline support: implement Progressive Web App features such as service workers and cached content so previously viewed trek information remains available without a connection - particularly relevant in remote trekking regions.
- Multi-language support: extend the interface and the assistant's replies to Nepali and other major source-market languages.
- Advanced AI features: explore image-based landmark recognition so a traveller can upload a photograph to identify a location or route, and consider a locally fine-tuned recommendation model once enough booking and preference data has accumulated.
- Analytics dashboard: give administrators usage and booking-trend analytics to inform which treks and destinations to feature more prominently.

References

[1] M. A. Camilleri and C. Troise, "Chatbot recommender systems in tourism: A systematic review and a benefit-cost analysis," in Proc. 8th Int. Conf. Machine Learning Technologies (ICMLT), Stockholm, Sweden, Mar. 2023, pp. 1-8.

[2] R. Alotaibi, A. Ali, H. Alharthi and R. Almehamdi, "AI chatbot for tourism recommendations: A case study in the city of Jeddah, Saudi Arabia," Int. J. Interactive Mobile Technologies, vol. 14, no. 19, pp. 18-30, 2020.

[3] G. Saluja, S. N. Maheswari, T. S. P. Kumar and M. Sivagami, "AI based intelligent travel chatbot for content oriented user queries," Int. J. Applied Science and Engineering, vol. 18, no. 2, pp. 1-8, 2021.

[4] "Review on study and usage of MERN stack for web development," Int. J. Research in Applied Science and Engineering Technology (IJRASET), 2022.

[5] "MERN stack: Technologies used for web development," Int. J. Research in Applied Science and Engineering Technology (IJRASET), 2023.

[6] M. B. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[7] MongoDB Inc., "MongoDB manual." [Online]. Available: https://www.mongodb.com/docs/. [Accessed: 20 Aug. 2026].

[8] OpenJS Foundation, "Express.js documentation." [Online]. Available: https://expressjs.com/. [Accessed: 20 Aug. 2026].

[9] Meta Open Source, "React documentation." [Online]. Available: https://react.dev/. [Accessed: 22 Aug.2026].

[10] Hugging Face, "Inference API documentation." [Online]. Available: https://huggingface.co/docs/api-inference/. [Accessed: 25 Aug. 2026].

[11] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson, 2016.

[12] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner’s Approach, 8th ed. New York, NY, USA: McGraw-Hill Education, 2014.

Appendix A: Screenshots of the System

The screenshots below show the finished platform: the landing page, the trek catalogue, a trek details page, the traveller's own booking list and the administrative dashboard. The two AI-assisted screens appear as Figures 4.1 and 4.2 in Chapter 4.

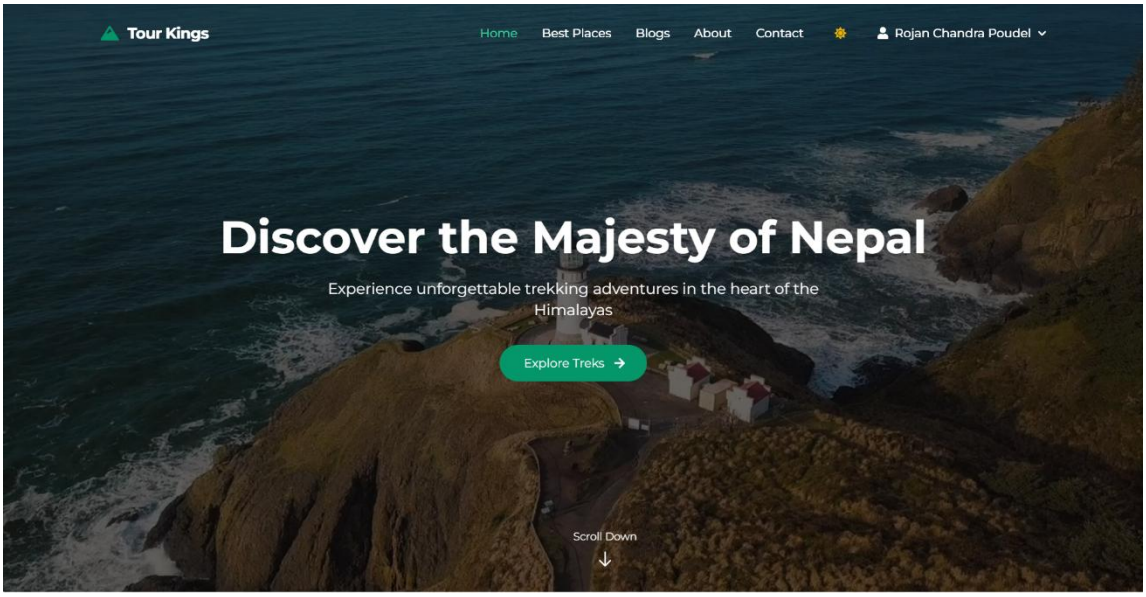


Figure A.1: Home page with the hero section and featured treks

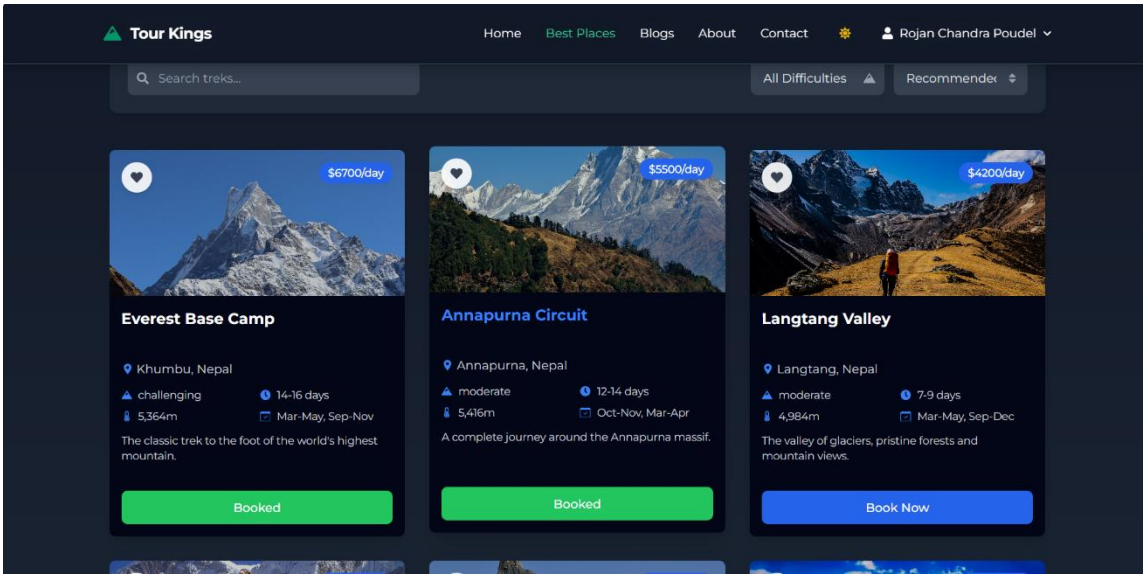


Figure A.2: Trek catalogue with difficulty, duration and budget filters



Figure A.3: Trek details page with the booking action

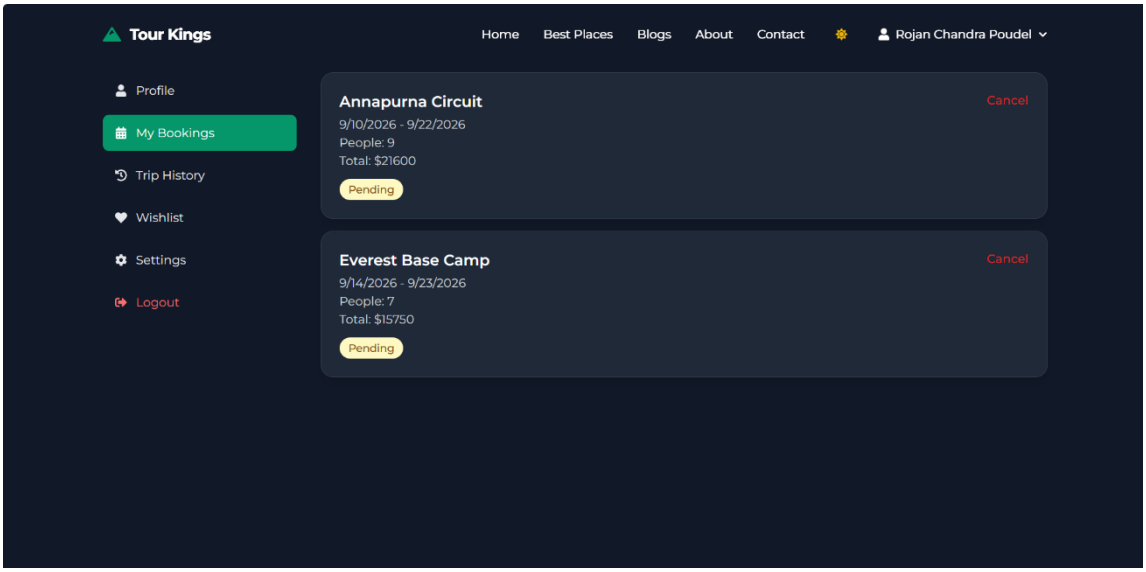


Figure A.4: My Bookings page showing each request and its status

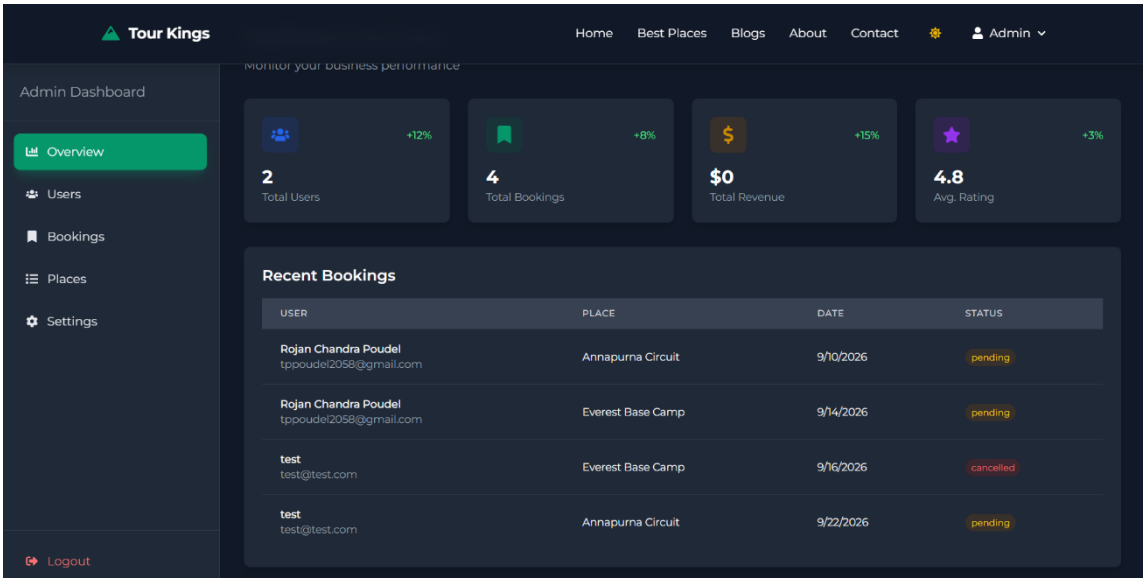


Figure A.5: Administrative dashboard with platform metrics and booking management